

AVL Tree Dictionary

A Self-Balancing Spell Checker in C. Built on an AVL (Adelson-Velsky and Landis) Tree - insertion, rotations, case-insensitive lookup, and nearest-match suggestions in $O(\log n)$.

1. Overview & Flow

This project is a console-based spell checker written in C, powered by a hand-built AVL Tree. Words are loaded from a dictionary file (Dictionary.txt) into the tree, one word per line. The program then reads a sentence typed by the user, splits it into words, and checks each word against the dictionary.

- **Load:** `load_file()` opens Dictionary.txt, reads it line by line, and calls `insert_node()` for every word, building a balanced AVL tree.
- **Report:** After loading, the program prints the total number of words and the tree's height.
- **Read & Check:** Asks the user to type a sentence, splits it using `strtok()`, and searches each word case-insensitively using `avl_search()`.
- **Correct/Incorrect:** Prints -CORRECT for exact matches. For incorrect words, it offers up to three suggestions: the closest node, its predecessor, and its successor.

2. Project Structure & Core Data Structure

File	Purpose
main.c	The full source code: AVL tree logic, spell-checking logic, and the program entry point.
Dictionary.txt	A plain text file with one valid word per line. Must sit in the same folder as the compiled executable.

The core data structure is the node. Every node stores one dictionary word (dynamically allocated), pointers to its children, and its subtree height. The height field is crucial for AVL balancing.

```
typedef struct node {
    char *data;
    struct node *right, *left;
    int hight;
} node;
```

3. Function Reference

Function	Description
init(s)	Allocates a new node, copies string 's', sets children to NULL and height to 1.
get_balance(x)	Returns height(left) - height(right). Used to detect imbalances.
rotate_right(x) / rotate_left(x)	Performs rotations to fix left-heavy or right-heavy imbalances.
insert_node(root, s)	Recursively inserts a word and rebalances the tree using LL, RR, LR, RL rotations.
search_node(root, s)	Iteratively searches for a word case-insensitively.
get_successor / get_predecessor	Finds the alphabetically immediate next/previous word.
avl_search(root, x)	Spell-checks 'x'. Prints CORRECT or suggests alternatives if missing.
load_file(filename)	Builds the dictionary tree from the text file.

4. AVL Balancing Explained

After every insertion, insert_node() recomputes heights and balance factors. One of four rotations is applied if the balance factor is outside [-1, 1]:

Case	Condition	Fix Applied
LL (Left-Left)	balance > 1 & inserted in left child's left subtree	Single rotate_right(root)
RR (Right-Right)	balance < -1 & inserted in right child's right subtree	Single rotate_left(root)
LR (Left-Right)	balance > 1 & inserted in left child's right subtree	rotate_left(root->left) then rotate_right(root)
RL (Right-Left)	balance < -1 & inserted in right child's left subtree	rotate_right(root->right) then rotate_left(root)

5. Time & Space Complexity

Operation	Time Complexity	Notes
Insert / Search	$O(\log n)$	Tree height is kept $\sim \log(n)$ via AVL balancing.
Predecessor / Successor	$O(\log n)$	Bounded by tree height.
Load dictionary (n words)	$O(n \log n)$	n insertions, each taking $O(\log n)$.
Space Complexity	$O(n)$	One node per word, plus dynamically allocated strings.

6. Building, Running & Limitations

To Compile and Run:

```
gcc main.c -o avl_dictionary
./avl_dictionary
```

Ensure Dictionary.txt is in the same directory as the executable.

Known Limitations

- **No Deletion:** Words can be added but not removed.
- **Suggestion Logic:** Suggestions are alphabetical, not phonetic or edit-distance (Levenshtein) based.
- **Memory Management:** Memory is not freed before exit (no `free_tree()` implemented).
- **Buffer Size:** Fixed buffer `char s[1000]` is used for input parsing.